Draft rows and the padding mask

A walkthrough of speculative decoding, the DFlash drafter in vllm-torchtpu, and commit `65da28e`, which refreshes the MoE padding mask before each draft forward.

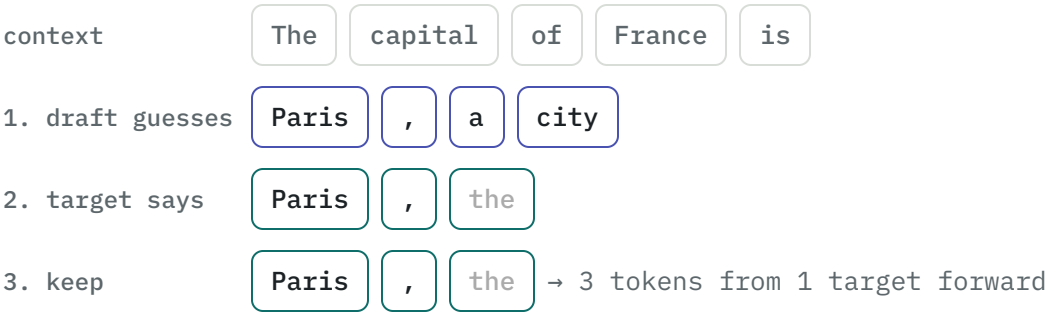
PART 1

Speculative decoding

Decoding one token at a time is memory-bound. Each step streams every weight of a large model from HBM to produce one token per request, and the matrix units are mostly idle. A forward over 4 tokens costs almost the same wall time as a forward over 1.

Speculative decoding uses that spare compute. A small, cheap **draft** model guesses the next K tokens. The big **target** model then checks all K guesses in a single forward.

One step, concretely (K = 4)



The target runs once on `[is, Paris, ",", a, city]`. Each position yields the target's own next-token prediction. Guesses are accepted left to right until the first mismatch. At the mismatch the target's own token ("the") is kept for free, and everything after it is thrown away.

- **Output is unchanged.** Every kept token is one the target itself would have produced (exactly so for greedy decoding; rejection sampling extends this to sampling).
- **Speed depends on the acceptance rate.** If the draft is good, each target forward yields up to K + 1 tokens. If it is bad, you get 1 token and paid for the draft.

- **A bad draft costs speed, not correctness.** This matters in Part 3: the bug there made drafts worse, so it hurt throughput without corrupting output.

PART 2

DFlash: guess K tokens in one draft forward

A classic drafter (EAGLE-style) is itself autoregressive. Guessing 4 tokens takes 4 sequential draft forwards. On TPU every forward has fixed launch and weight-streaming cost, so those draft steps add up.

DFlash drafts the whole block in **one** forward. For each request it builds a block of `K + 1` input slots: the last verified token (the "anchor"), followed by K `MASK` placeholders. Attention inside the block is non-causal, and the model fills in every mask slot at once.

Autoregressive drafter, K = 3

```
draft fwd 1 → t1
      draft fwd 2 → t2
            draft fwd 3 → t3
                  verify
```

DFlash, K = 3

```
draft fwd → t1 t2 t3
      verify
```

One draft forward over a slightly wider block replaces K sequential ones.

The draft input, concretely

Take K = 3, so `block_size = K + 1 = 4`. Two requests whose last verified tokens are `is` (position 41) and `def` (position 207):

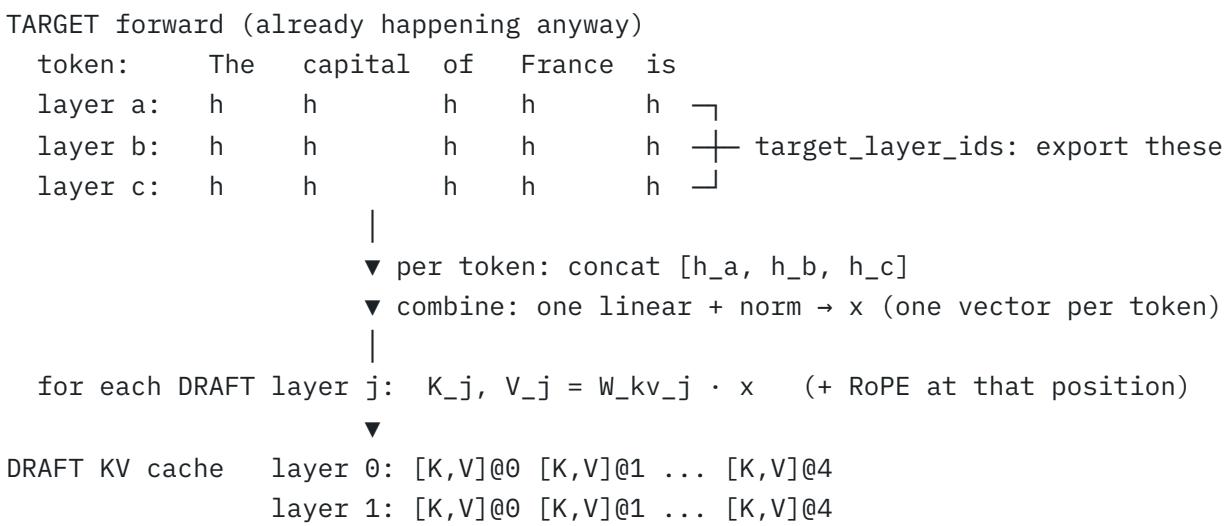


2 requests × 4 slots = **8 real rows**. Slot 0's output is dropped and slots 1..K become the K drafts (`_dflash_forward_and_sample`, `hidden_reshaped[:, 1:, :]`).

Where does the draft's context come from?

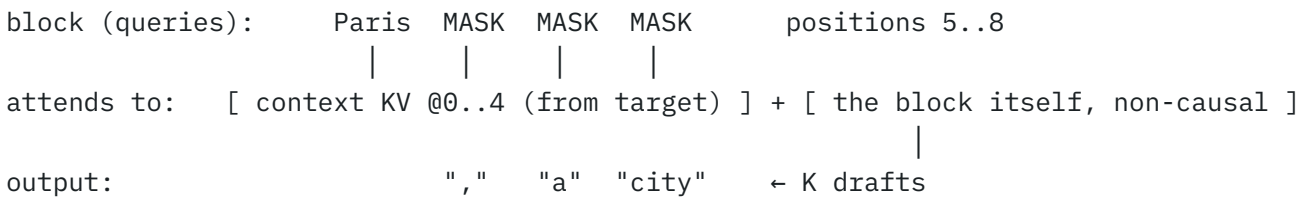
The problem. To guess the next tokens, the draft has to attend to the whole conversation so far. Normally that means the draft keeps its own KV cache, filled by running the draft's own layers over every context token: a full draft prefill, then another draft pass for each newly accepted token.

The trick. The target has already run over every context token, and its hidden states are a rich summary of each one. DFlash builds the draft's KV cache from them. Take the prompt `"The capital of France is"` (positions 0–4):



Each context token costs only one small projection per draft layer, not a full run through the draft's layers. The draft never runs attention or an MLP over the context.

The draft forward only processes the block. Say the target sampled "Paris" as the next token:



So each step the draft does real compute on only `n_reqs × (K+1)` rows, however long the conversation is.

Why "after each target forward". Every step, the target processes some new tokens. `_update_draft_kv_cache_from_target` writes those tokens' entries into the draft's cache before the draft forward runs. That's the "Hoist all KV cache updates" loop at the top of `propose()`.

Why it's designed this way. It is cheap, because it skips running the draft over the context. It also gives better guesses, because the draft conditions on the big model's internal features rather than just token ids, which helps acceptance.

Code: `dflash.py` `_update_draft_kv_cache_from_target` and `_tpu_precompute_and_update_kv_cache`. DSv4 DSpark: `models/vllm/deepseek_v4/dspark.py` `precompute_and_store_context_kv` (`main_norm(main_proj(concat))`, then each layer's `fused_wqa_wkv` → `kv_norm` → cache). Kimi K3 DSpark: one fused `context_kv_proj` outputs the KV for all draft layers at once. Not covered here: how cache entries for rejected draft positions get overwritten.

DSpark (`spec_decode/dspark.py`) subclasses this proposer. It uses the same parallel block forward and adds a small sequential "Markov head" that conditions each position on the previously sampled draft token. For the DeepSeek V4 and Kimi K3 checkpoints, the DSpark draft layers are **MoE layers**. That is the key fact for Part 3.

PART 2B

Reading dflash.py

The file is one class, `DFlashProposer` . It does three jobs: **set up** the draft model once, **propose** drafts every step, and **warm up / replay** with dummy inputs.

```
┌──────────── once at startup ────────────┐
│ __init__      read config, pick TP      │
│ load_model    load draft, wire it up    │
│ precompile    compile every shape       │
└─────────────────────────────────────────┘
```

```
every step, after the target forward:
  propose()
    1. KV update      target hidden states → draft KV cache
    2. build inputs   [anchor, MASK, MASK, MASK] per request
    3. draft forward  one pass → K draft tokens per request
    4. return         the drafts (target verifies them next step)
```

1. Setup

- `__init__` reads the draft config: `mask_token_id` (the MASK placeholder) and `target_layer_ids` (which target layers to borrow hidden states from). It picks how the draft is split across chips: only **replicated** (TP = 1) or **sharded like the target**.

- `block_size` is `K + 1`: one anchor plus K MASK slots. DSpark overrides it to K.
- `load_model` tells the target to export hidden states from `target_layer_ids`, loads the draft, shares the target's embedding table and LM head (same vocabulary), and sets the draft's attention to **non-causal** so the MASK slots see each other inside the block.

2. `propose()`, the per-step path

Step 1, KV update (`_update_draft_kv_cache_from_target` → `_tpu_precompute_and_update_kv_cache`). This is the trick from "Where does the draft's context come from?": target hidden states → concat → combine → K/V for every draft layer → draft KV cache. All draft layers are done in one fused projection.

Step 2, build inputs (`_prepare_dflash_inputs`). Per request it builds `[anchor, MASK, MASK, MASK]` at positions `[p, p+1, p+2, p+3]`, then pads to a bucket. There are two variants, depending on what the target just did:

- `_tpu_build_dflash_inputs_next_tokens`: the target just verified drafts. The anchor is the **last accepted token**, and `p` = start + number accepted.
- `_tpu_build_dflash_inputs_seed`: the target just did prefill or a step with no drafts. The anchor is the **just-sampled token**, and `p` = last prompt position + 1.

Both share `_layout_draft_block` for the layout itself.

Step 3, draft forward + sample (`_dflash_forward_and_sample`). First `mark_padding(...)` runs (this commit). Then: draft model → drop slot 0 (the anchor) → logits for the K slots → argmax → K drafts. It runs as one compiled XLA program.

Step 4, return. The drafts per chunk go back to the runner, which feeds them into the next target step.

3. Dummy runs (no real requests)

FUNCTION	WHEN	WHY
`precompile`	startup	Run every bucket size once, so XLA never compiles mid-serving
`_dummy_precompute_and_update_kv_cache`	precompile and DP	KV update on zeros

FUNCTION	WHEN	WHY
`_dummy_draft_forward`	precompile and DP	Draft forward on zeros (the other `mark_padding` call)
`run_dp_dummy_draft`, `_dp_dummy_*`	data parallel	An idle chip must issue the **same collectives** as busy chips, or the busy ones hang

The two ideas behind most of the code

- **Static shapes.** XLA compiles one program per shape. That's why everything is padded to buckets, uses cached tensors (`_get_static_attn_tensors` , `_zero_draft_lengths`), and gets precompiled.
- **DP lockstep.** With data parallelism and an MoE draft, every chip must run the same collectives in the same order. That's why there are extra dummy chunks, `_dp_draft_bucket` , and the rule "all KV updates before all draft forwards".

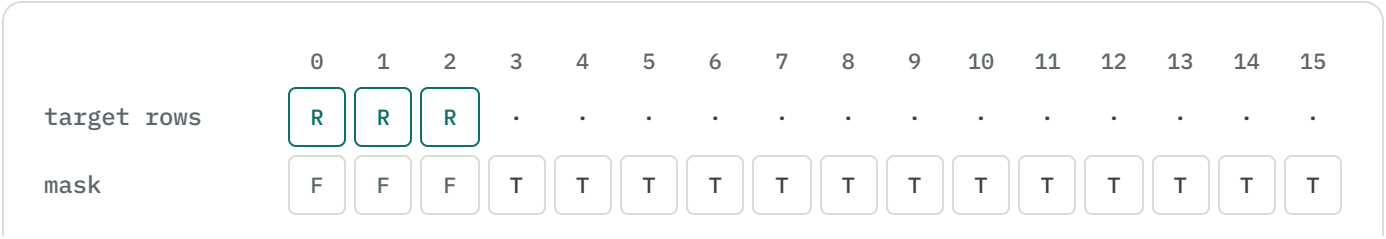
Strip those two away and DFlash is about three functions: the KV update, the input layout, and the forward-and-sample.

PART 3

Padding rows and the MoE padding mask

XLA compiles one program per input shape. To avoid recompiling for every batch size, the runner pads the token count up to a fixed **bucket** (8, 16, 32, ...). The extra rows are garbage and their outputs are discarded.

In a dense layer, padding rows cost FLOPs but nothing else. In an MoE layer they also get routed to experts, which adds expert matmuls and, under expert parallelism, cross-chip traffic. So with `TPU_MOE_SKIP_PADDED_TOKENS=1` the runner keeps a per-rank boolean **padding mask**, and MoE layers skip the rows marked `True`:



Target step with 3 real tokens in a 16-row bucket. Rows 3–15 are marked padding and skip expert compute.

MOE PATH	HOW A MASKED ROW IS SKIPPED
FP8 GMM (`fp8.py`)	top-k routing weights set to 0
Fused EP (`fused_moe_ep.py`)	router logits set to NaN → row is "unrouted" and skipped by scatters, DMAs and matmuls
K3 W4A-MXFP4 (`compressed_tensors_moe_w4an_mxfp4.py`)	routing weights set to 0, *whenever the mask exists, even without the env flag*

The mask buffer is created once in `tpu_runner.py` (around line 5080) and stored in a module-level global, `token_padding._padding_state`.

The bug: one mask, two models

There is only one **mask buffer** per chip: an array of 16 booleans, where `T` means "this row is padding, skip it in the MoE". The target and the draft both run on the chip and both read this buffer. Before this commit, only the target ever wrote it (`_token_padding_update`). Here is what happens in one step (case A below):

Step 1: target forward. First decode step after prefill: 2 requests, no drafts to verify yet, so 1 row each = 2 real rows, padded to 16. Before running, it writes the mask buffer:

```
mask buffer:   F F T T T T T T T T T T T T T T
               └2┘ (rows 2-15 are padding)
```

This is correct for the target.

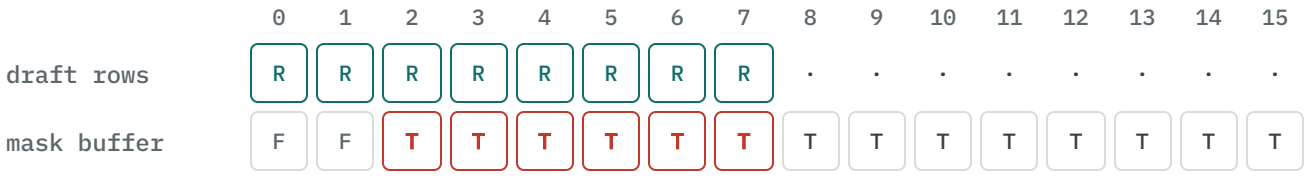
Step 2: draft forward, same step, same chip. The draft's input is different: 2 requests × 4 slots = 8 real rows, padded to 16. Before the fix, nobody rewrote the mask buffer here, so the draft's MoE layers read it exactly as step 1 left it:

```
draft rows:    R R R R R R R R · · · · · · · ·
mask buffer:   F F T T T T T T T T T T T T T T
                   ^^^^^^^^^^
               rows 2-7: real draft tokens, but the mask says "padding"
```

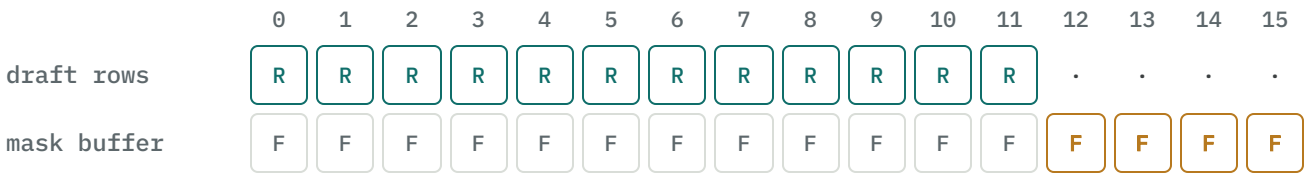
Result. The MoE skips rows 2–7, so those tokens get no expert output. Their hidden states are wrong, so their draft guesses are likely wrong too. The target then rejects more drafts. Output stays correct, but you get fewer tokens per step.

> **In one sentence:** the mask is per-row information ("which rows are padding"). The target and the draft have different rows, but only the target ever wrote the mask, so the draft used the target's answer for its own rows.

In a normal verify step both sides have `n_reqs × (K+1)` rows, so the mask happens to match. It breaks when the target's rows differ, either way:



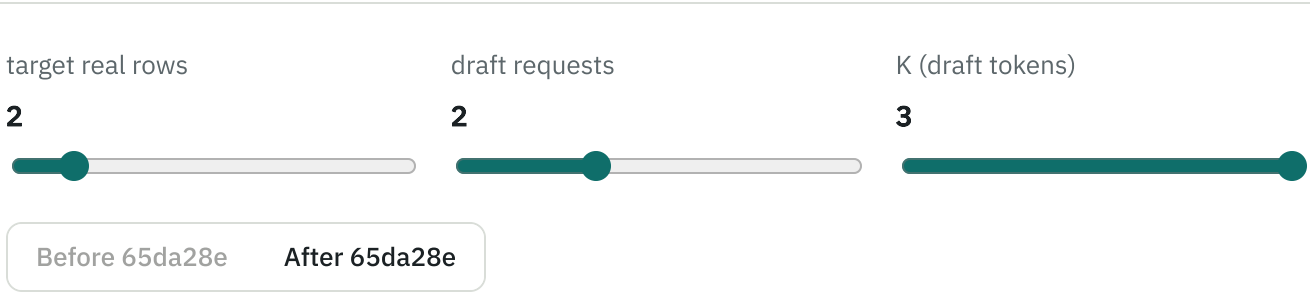
Case A: no drafts to verify (first decode after prefill). Target 2 rows, draft 8. Rows 2–7 lose their experts, so acceptance drops.

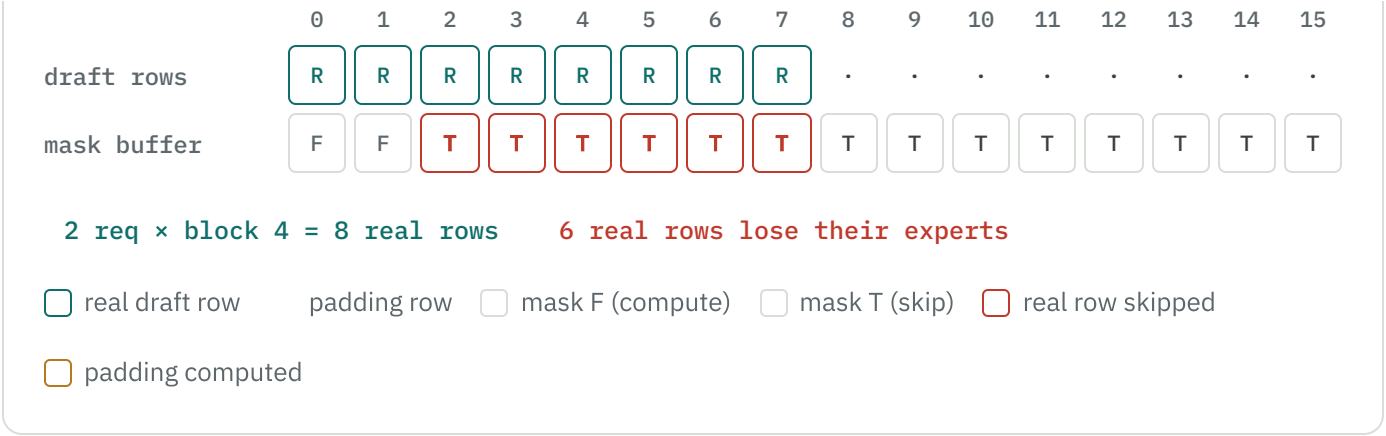


Case B: the step includes a prefill. Target 16 rows (an 8-token prefill + 2 × 4 verify rows), draft 3 requests × 4 = 12. Padding rows 12–15 run through the experts for nothing.

Mask playground

Change the target's real-row count and the draft's request count, then switch between the old and new behavior. Bucket is 16 rows, K = 3 (block 4).





The fix

Before each draft forward, rewrite the mask to describe the draft's own rows. That takes a new helper and two calls:

```
# layers/adapter/token_padding.py
def mark_padding(num_valid_tokens, num_padded_tokens):
    if _padding_state is not None:
        _padding_state.update(num_valid_tokens, num_padded_tokens)

# spec_decode/dflash.py, propose(), once per chunk
token_padding.mark_padding(chunk.num_reqs * block_size, padded_len)
#   2 reqs × 4 = 8 real rows, bucket 16  →  F F F F F F F F T T T T T T T T

# spec_decode/dflash.py, _dummy_draft_forward()
token_padding.mark_padding(0, num_tokens)
#   every row is padding               →  T T T T T T T T
```

The dummy forward covers three cases where a rank runs the draft with no real requests: DP idle replay (an idle rank must still issue the same EP collectives as busy ranks), extra lockstep chunks, and precompile. Marking every row as padding there matches what the runner already does for its own warmup steps (`tpu_runner.py`, around line 5397).

The new mask never leaks back into the target: the target rewrites the mask before each of its own forwards.

The commit adds two tests in `tests/spec_decode/test_dflash.py`. Each starts with the mask buffer left in a "3 of 16" target state and asserts what the draft forward sees: `[F]*8 + [T]*8` in `propose()`, and `[T]*8` in the dummy forward.

REVIEW

Is the change right?

Yes. Checked against the source:

- **The real-row count is correct.** `_prepare_dflash_inputs` puts `num_reqs × block_size` real rows first and pads the rest up to `padded_len`, so `[num_reqs·block_size, padded_len)` is exactly the padding range.
- **It is per chunk.** The call sits inside the chunk loop, so each chunk's draft forward gets its own mask.
- **No recompiles.** The mask is a device tensor's *contents*, not a shape, so changing it does not change the compiled program.
- **The cost is small but nonzero.** `update()` does a small synchronous host-to-device copy (a bool vector of `max_num_tokens`) per draft forward. The target already pays this once per step. I have not measured it.

> **Takeaway:** the padding mask is per-rank global state. Any forward whose row layout differs from the target step's has to call `mark_padding` before it runs. Otherwise its MoE layers silently trust someone else's rows.

Sources: `src/vllm_torchtpu/layers/adapter/token_padding.py`, `src/vllm_torchtpu/spec_decode/dflash.py`, `src/vllm_torchtpu/spec_decode/dspark.py`, `src/vllm_torchtpu/runner/tpu_runner.py`. The DFlash description here comes from this repo's implementation, not from the original paper.